DIGITAL EXPENSE TRACKER

A PROJECT REPORT

Submitted by

Avtar Singh(23BCS13396)

Devesh Rajpoot(23BCS13411)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

Chandigarh University

April 2026

BONAFIDE CERTIFICATE

Certified that this project report titled "**DIGITAL EXPENSE TRACKER"** is the bonafide work of the "**Avtar Singh and Devesh Rajpoot"** who carried out the project work under my guidance and supervision.

The work presented here is based on the submitted paper and has been rewritten in project report form by following the university mini project structure. The report explains the idea, method, results, and practical use of the Vision Transformer model in simple and clear language.

Signatures and official details may be completed by the department before final submission.

SIGNATURE	**SIGNATURE**
HEAD OF THE DEPARTMENT	**SUPERVISOR**

ACKNOWLEDGEMENT

The successful completion of the mini project Digital Expense Tracker was possible because of the guidance, encouragement, and support extended by many individuals. We sincerely thank our project supervisor for valuable mentoring and feedback during the planning, development, testing, and documentation stages of this work.

We are thankful to the Head of the Department, faculty members, friends, and family for their encouragement. Feedback from student users living in hostels, rented rooms, and paying guest accommodations helped us identify practical requirements such as quick expense entry, monthly budget tracking, and dashboard-based analysis.

We also thank Chandigarh University for providing the academic environment and resources required to undertake this mini project and present a software solution focused on student financial awareness.

TABLE OF CONTENTS

S. No.	Section Title	Page No.
1	List of Figures	i
2	List of Tables	i
3	Abstract	ii
4	Graphical Abstract	iii
5	Abbreviations and Symbols	iii
6	Chapter 1. Introduction	1
6.1	Client / Need Identification	1
6.2	Identification of Problem	2
6.3	Identification of Tasks	2
6.4	Timeline	3
6.5	Organization of the Report	3
7	Chapter 2. Design Flow / Process	4
7.1	Concept Generation and Features	4
7.2	Selection of Specifications / Features	5
7.3	Design Constraints	5
7.4	Feature Finalization	6
7.5	Alternative Designs and Selection	6
7.6	Implementation Plan / Methodology	7
8	Chapter 3. Results Analysis and Validation	8
8.1	Implementation of the Solution	8

8.2	Testing and Validation	9
9	Chapter 4. Conclusion and Future Work	10
9.1	Conclusion	10
9.2	Future Work	11
10	References	12
11	Appendix – User Manual	13
12	Achievements	13

LIST OF FIGURES

Figure 1.1 Problem context for students living away from home 1

Figure 1.2 High-level use case view 2

Figure 1.3 Timeline / Gantt chart 3

Figure 2.1 Overall system architecture 4

Figure 2.2 Alternative design A – Client-heavy flow 6

Figure 2.3 Alternative design B – API-centric flow 6

Figure 2.4 Final application workflow 7

Figure 3.1 Dashboard and monthly analytics screen 8

Figure 3.2 Expense entry and category distribution 9

Figure A.1 Login / registration interface 13

Figure A.2 Add expense workflow 13

LIST OF TABLES

Table 1.1 Need analysis derived from student pain points 1

Table 2.1 Functional and non-functional specifications 5

Table 2.2 Design constraints considered in the project 5

Table 2.3 Comparison of alternative designs 6

Table 2.4 Technology stack and engineering tools 7

Table 3.1 Core modules implemented in the application 8

Table 3.2 Sample validation test cases 9

Table 4.1 Planned enhancements 11

ABSTRACT

Digital Expense Tracker is a web-based mini project developed to help college students monitor and control daily spending, especially those living outside their homes for higher education. Students commonly spend money on rent, food, transport, stationery, mobile recharge, medicine, and emergency purchases, but many do not maintain structured records. This leads to poor budget control and low awareness of spending patterns.

The proposed system provides a simple platform for recording daily expenses, categorizing transactions, setting monthly budgets, and visualizing expenditure patterns on an interactive dashboard. The frontend is implemented using React for a responsive interface. The backend is developed using Java Spring Boot, which exposes REST APIs for authentication, expense management, dashboard analytics, and budget tracking. MySQL is used for persistent data storage and supports normalized transaction records, user profiles, categories, and budget-related entries.

The project follows an engineering design approach in which the need was identified through the lifestyle patterns of students residing in hostels, paying guest accommodations, and rented rooms. The implemented solution supports secure login, expense entry, editing, deletion, category-wise filtering, monthly summary, and overspending alerts. Validation through module-level testing and user-oriented scenarios confirmed correctness, usability, and practical relevance.

GRAPHICAL ABSTRACT

Student Need	Expense Entry	Spring Boot API	MySQL Storage	Analytics
Untracked daily spending outside home	Daily food, rent, travel and utility entries	Validation, business rules and reporting services	Persistent storage for users, budgets and expenses	Dashboard trends and budget alerts

Input → validation and categorization → storage → dashboard insight → improved budgeting behaviour.

Figure G.1 Conceptual flow of the Digital Expense Tracker.

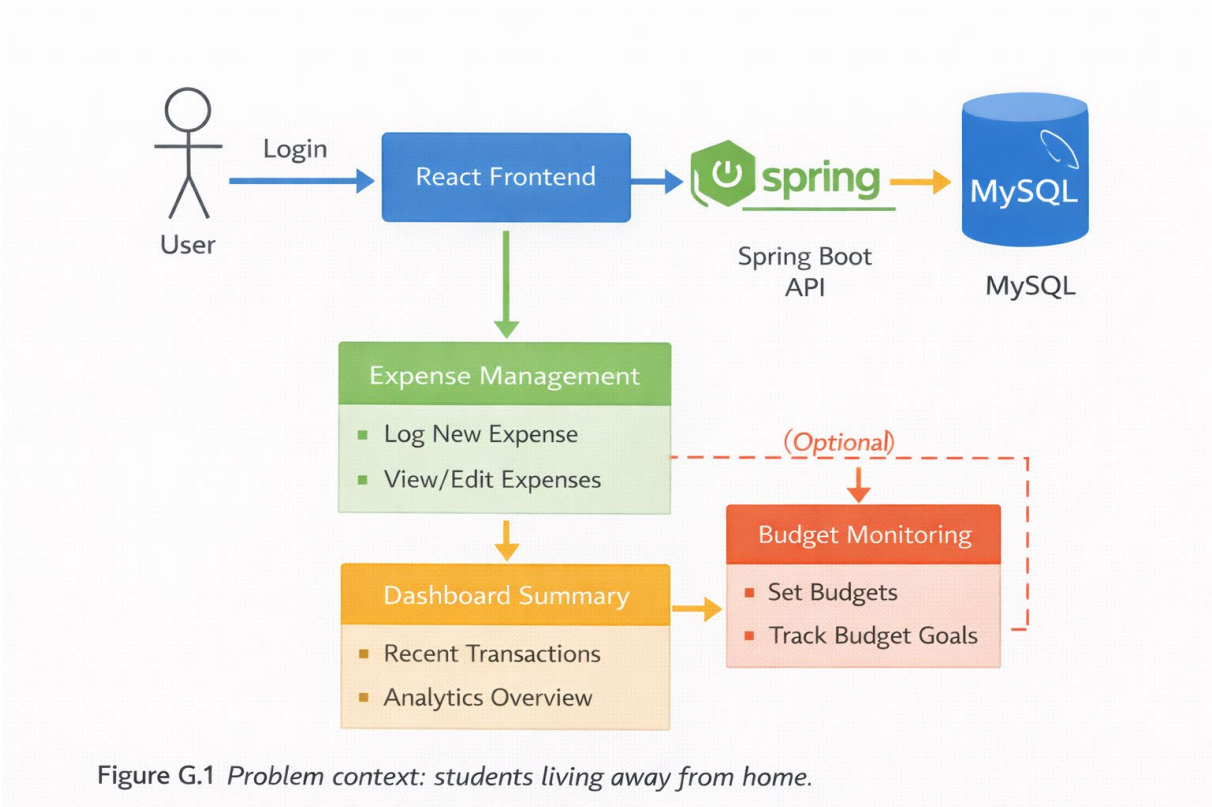

Figure G.1 *Problem context: students living away from home.*

ABBREVIATIONS AND SYMBOLS

Abbreviation	Expanded Form	Meaning in Project
UI	User Interface	Screens visible to the student user
API	Application Programming Interface	REST endpoints exposed by Spring Boot
DB	Database	Persistent storage implemented using MySQL
CRUD	Create, Read, Update, Delete	Basic data operations on expenses
JWT	JSON Web Token	Token-based authentication option
Rs.	Indian Rupees	Currency symbol for amounts

CHAPTER 1

INTRODUCTION

Students who live outside their homes for college often manage a fixed monthly allowance or limited part-time income. Their routine includes recurring and variable expenses such as meals, rent contribution, transport, internet recharge, stationery, medicine, laundry, and occasional emergencies. Because these transactions are small and frequent, they are often not recorded systematically, leading to poor visibility of actual spending.

The primary client for the project is therefore the college student who needs a lightweight digital tool to capture and monitor expenses in real time. The issue is contemporary because cashless payments and app-based services have increased transaction frequency while informal tracking methods remain common.

Table 1.1 Need analysis derived from student pain points

Observed Pain Point	Practical Effect	Required System Response
Small cashless transactions are forgotten	Monthly spending appears lower than actual	Quick daily entry with category
No view of category totals	Food or transport overspending goes unnoticed	Category-wise totals and charts
No budget benchmark	Student cannot compare spend vs target	Monthly budget setup and alerts
Scattered manual records	Past spending is difficult to review	Central database history

Figure 1.1 Problem context for students living away from home.

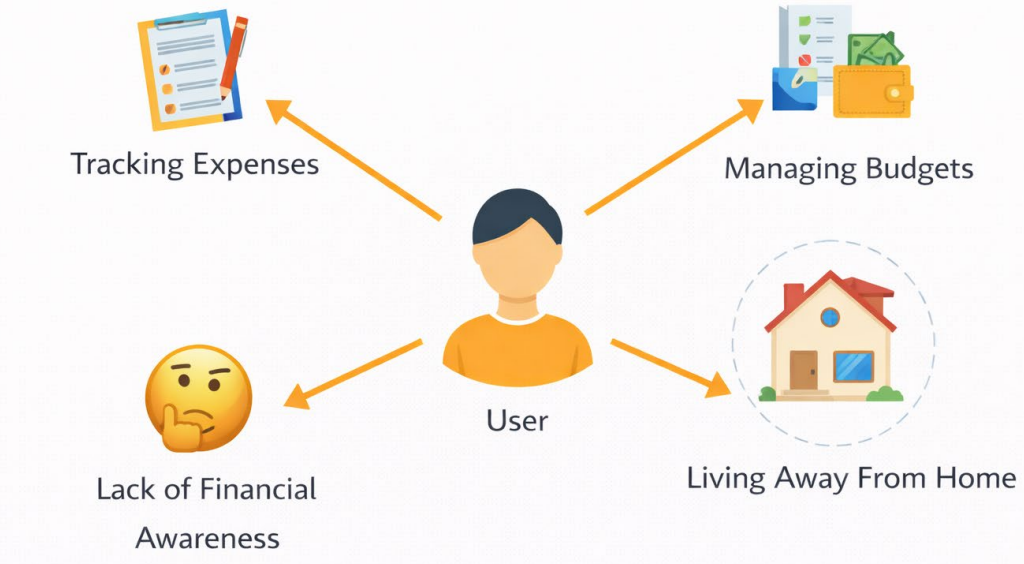

Figure 1.1 *Problem context: students living away from home.*

Identification of Problem

The broad problem addressed in this mini project is the absence of a focused, user-friendly digital mechanism for students to record and analyze daily expenses while living outside their homes. This causes low financial visibility, weak categorization of spending, and poor month-end planning.

Identification of Tasks

The project was divided into tasks covering need analysis, system planning, frontend development in React, backend development in Spring Boot, database design in MySQL, integration, testing, and report preparation.

Figure 1.2 High-level use case view: login, expense logging, dashboard review, and budget monitoring.

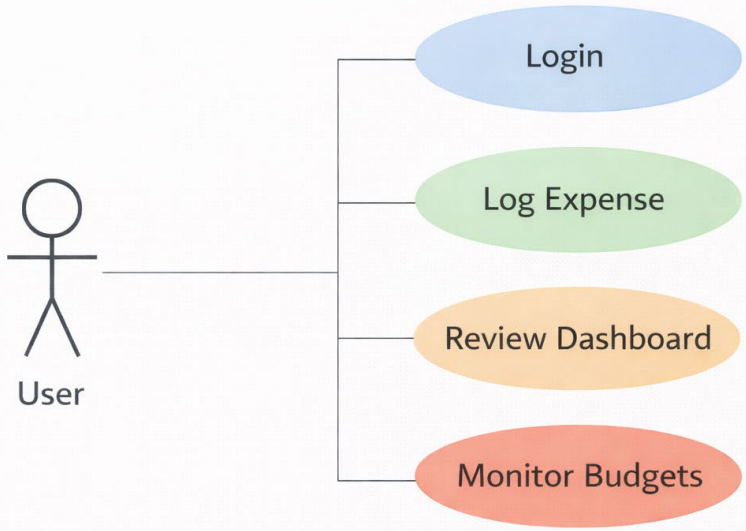

Figure 1.2 *High-lewel use case view: Login, expense logging, dashboard review, and budget*

Timeline

Task	W1	W2	W3	W4	W5
Need analysis & features	●	●			
Architecture & DB design		●	●		
Frontend in React			●	●	
Backend in Spring Boot			●	●	●
Integration & testing				●	●
Report writing					●

Figure 1.3 Semester-wise timeline / Gantt chart equivalent.

Figure 1.3 *Week-wise timeline / Gantt chart equivalent.*

Organization of the Report

Chapter 1 introduces the problem context and need. Chapter 2 presents the design process, constraints, alternatives, and methodology. Chapter 3 explains implementation, testing, and validation. Chapter 4 concludes the work and lists future enhancements. References and a user manual are included at the end.

CHAPTER 2

DESIGN FLOW / PROCESS

2.1 Concept Generation and Literature-Informed Features

Concept generation started with the study of common expense tracking approaches such as paper notes, spreadsheets, generic budgeting apps, and web portals. For student users, the most important expectations were speed of entry, category clarity, and meaningful visualization rather than enterprise-grade accounting features.

Three concept options were considered: a spreadsheet-only system, a mobile-first offline application, and a web-based application with clear frontend-backend separation. The web-based option was selected because it supports modular development, easier testing, and future extensibility.

Figure 2.1 Overall system architecture: React frontend communicates with Spring Boot APIs, which interact with MySQL storage.

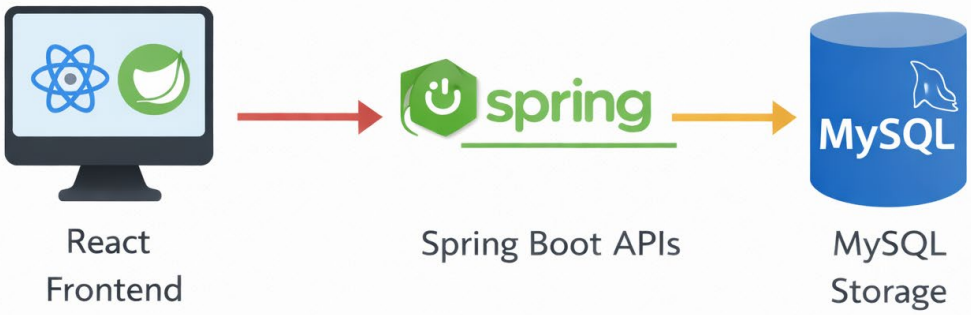

Figure 2.1 *Overall system archirecture: React frontend communicates with Spring Boot APIs; which interact with MySQL storage.*

2.2 Evaluation & Selection of Specifications / Features

The selected features combine functional usefulness with usability and maintainability.

Table 2.1 Functional and non-functional specifications

Type	Specification	Reason for Selection
Functional	Registration and login	Personalized tracking
Functional	Add, edit, delete expense	Complete CRUD operations
Functional	Category-wise classification	Better spending analysis
Functional	Monthly budget and alerts	Promotes financial discipline
Functional	Dashboard and reports	Converts raw data into insight
Non-functional	Responsive interface	Useful on browser and phone
Non-functional	Secure API flow	Protects user data
Non-functional	Scalable backend	Supports future expansion

2.3 Design Constraints

The final design was shaped by economic, technical, social, and professional constraints.

Table 2.2 Design constraints considered in the project

Constraint	Impact on Design	Response in Final System
Economic	Students need a low-cost solution	Browser-based stack avoids special hardware
Implementability	Mini project time is limited	Used React, Spring Boot, and MySQL
Security	Financial data should not be exposed	Validation and controlled API access
Ethical	Expense data is personal	Restricted visibility to logged-in user
Social	Application must fit student habits	Fast entry and simple categories

2.4 Analysis and Feature Finalization subject to Constraints

Advanced forecasting, receipt scanning, and shared group settlement were identified as desirable but were deferred because they would increase scope and complexity. The final feature set therefore focused on secure login, expense CRUD operations, categories, monthly summaries, budget comparison, and dashboard analytics.

2.5 Alternative Designs and Design Selection

Two alternatives were studied. Design A handled most logic in the frontend, whereas Design B used an API-centric architecture with centralized business logic in Spring Boot. Design B was selected because it offers better consistency, scalability, and future integration support.

Table 2.3 Comparison of alternative designs

Criterion	Design A: Client-heavy	Design B: API-centric	Selected
Validation consistency	Medium	High	B
Security control	Limited	Centralized	B

Scalability	Moderate	High	B
Future integration	Difficult	Easier	B

Figure 2.2 and Figure 2.3 represent the conceptual client-heavy and API-centric flows respectively.

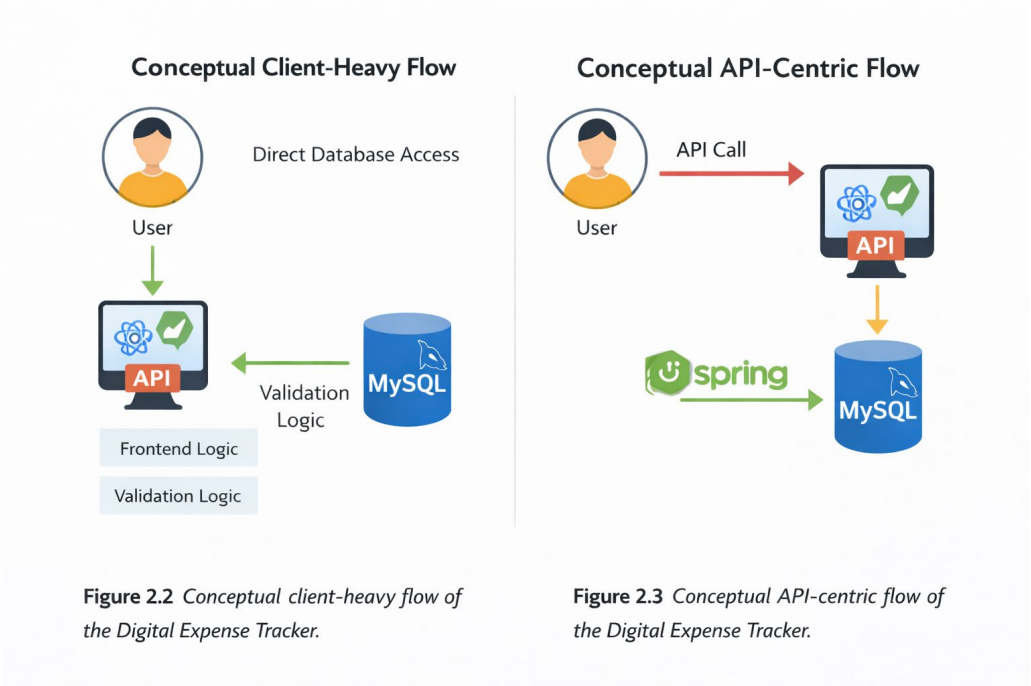

Figure 2.2 *Conceptual client-heavy flow of the Digital Expense Tracker.*

Figure 2.3 *Conceptual API-centric flow of the Digital Expense Tracker.*

2.6 Implementation Plan / Methodology

The implementation plan followed a layered approach: React components for presentation and state-driven interaction, service calls for API communication, Spring Boot controllers/services/repositories for business logic, and MySQL tables for persistent storage and reporting.

Table 2.4 Technology stack and engineering tools

Layer / Tool	Technology Used	Purpose
Frontend	React	Responsive interface and forms
Backend	Java Spring Boot	REST API and business logic
Database	MySQL	Persistent storage
Testing	Postman, browser validation	API and workflow checks
Versioning	Git / GitHub	Code tracking and project management

Figure 2.4 Final application workflow: user action → API call → validation/service → MySQL update/query → response.

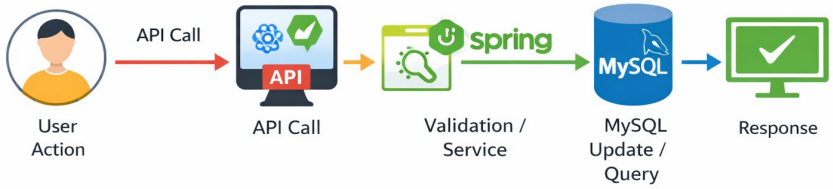

Figure 2.4 *Final application workflow:* user *action* → API call →*validation/service* → *MySQL update/query* → *response.*

CHAPTER 3

RESULTS ANALYSIS AND VALIDATION

3.1 Implementation of the Solution

The implemented Digital Expense Tracker uses a three-layer web architecture. The React frontend contains pages for authentication, expense entry, expense history, filtering, dashboard charts, and budget summaries. Forms are designed for quick daily logging with fields such as amount, date, category, and description.

Java Spring Boot provides REST endpoints that process authentication, transaction creation, record updates, filtered retrieval, and monthly aggregation. Service classes apply validation rules and repository classes interact with MySQL through a normalized schema for users, categories, budgets, and expenses.

From a user perspective, the system reduces uncertainty in day-to-day financial behaviour. Instead of relying on memory, the student can inspect exact transaction history and identify expensive habits such as frequent food delivery, repeated transport costs, or unplanned purchases.

Table 3.1 Core modules implemented in the application

Module	Key Functions	Outcome
Authentication	Register, login, session validation	Personalized secure access
Expense Management	Add, edit, delete, list expenses	Complete record handling
Category Management	Food, travel, rent, utilities, misc.	Meaningful classification
Dashboard Analytics	Monthly total, category split, recent trends	Better spending visibility
Budget Module	Set limit and compare actual spending	Alert for overspending

Figure 3.1 Dashboard and monthly analytics screen.

Figure 3.1 *Dashboard and monthly analytics screen.*

Figure 3.2 Expense entry and category distribution.

3.2 Testing, Characterization and Validation

Validation focused on correctness of data handling, consistency of dashboard results, and usability of the student workflow. Functional tests were performed for login, adding an expense, editing a transaction, deleting a transaction, category-wise filtering, monthly summary generation, and budget comparison.

Representative scenarios included recording a hostel meal expense, adding a monthly room-rent contribution, reviewing transport costs for a week, and checking whether the monthly food budget had been exceeded. In each case, the system produced structured records and corresponding analytical summaries.

Table 3.2 Sample validation test cases

Test Scenario	Expected Result	Observed Result	Status
Valid user login	Dashboard opens for authenticated user	Dashboard loaded successfully	Pass
Add daily food expense	New transaction stored and shown in list	Expense added and reflected in total	Pass
Edit transport expense	Updated amount visible in history and dashboard	Data updated correctly	Pass
Delete duplicate entry	Selected record removed from list and DB	Deletion successful	Pass
Budget comparison	Overspend warning beyond set limit	Warning displayed as expected	Pass

The project met its intended outcome of improving financial visibility for student users. A minor deviation from the ideal vision is that the current version depends on manual entry and does not yet include receipt capture or bank synchronization. However, this limitation is acceptable within mini project scope and preserves simplicity for the target users.

CHAPTER 4

CONCLUSION AND FUTURE WORK

4.1 Conclusion

The Digital Expense Tracker successfully addresses a real problem faced by college students living away from home. By combining React at the frontend, Java Spring Boot at the backend, and MySQL for storage, the project demonstrates a clean full-stack implementation aligned with modern software engineering practices.

The application enables students to record daily expenditure, classify spending by category, review monthly summaries, and compare actual expenses against budget expectations. The expected outcome of improving financial visibility and decision making was achieved through simple interaction flows and dashboard-based analysis.

4.2 Future Work

Future work can extend the project without changing its student-centric focus.

Table 4.1 Planned enhancements

Enhancement	Purpose	Benefit
Receipt image scan or OCR support	Reduce manual data entry	Improves convenience
Predictive monthly budgeting	Estimate likely end-of-month spend	Supports proactive decisions
Recurring expense templates	Auto-add rent or subscriptions	Saves repeated effort
Export to PDF/Excel	Generate shareable reports	Improves record keeping
PWA/mobile packaging	Better access on phones	Enhances adoption

Further improvements may also include notifications, multilingual support, and stronger security mechanisms. These additions can evolve the mini project into a robust student financial awareness platform.

REFERENCES

Freeman, E. and Robson, E. (2014) Head First Design Patterns, O'Reilly Media.

Banks, A. and Porcello, E. (2020) Learning React, O'Reilly Media.

Walls, C. (2022) Spring Boot in Action, Manning Publications.

Elmasri, R. and Navathe, S.B. (2016) Fundamentals of Database Systems, Pearson.

Oracle (2025) Java Documentation and Language Guides.

React Documentation (2025) Building User Interfaces with Components.

Spring Documentation (2025) Spring Boot Reference Guide.

MySQL Documentation (2025) MySQL 8.0 Reference Manual.

APPENDIX

USER MANUAL

Install Node.js and npm for the React frontend, Java Development Kit, Maven or Gradle for Spring Boot, and MySQL Server for database storage.

Create the required MySQL database and configure database credentials in the Spring Boot application properties file.

Run the Spring Boot backend application so that REST APIs are available locally.

Open the React frontend project, install dependencies using npm install, and start the application using npm start.

Register a new account or log in using existing credentials, then set a monthly budget.

Add expense records by entering amount, date, category, and description. Review the dashboard and edit or delete incorrect entries when needed.

Figure A.1 Login / registration interface – allows the student to create an account and begin personal expense tracking.

Figure A.2 Add expense workflow – a form-driven process for storing daily transactions quickly and accurately.

ACHIEVEMENTS

Designed a complete full-stack expense tracking application using React, Spring Boot, and MySQL.

Converted an everyday student financial problem into a structured engineering problem with measurable system features.

Implemented CRUD operations, dashboard analytics, category-wise tracking, and monthly budgeting support.

Demonstrated modular software architecture and practical use of modern engineering tools.

Prepared a report, manual, and validation artifacts aligned with the prescribed university format.

This mini project provides a practical foundation for future academic extension and deployment-oriented enhancement.